

Documentation recherche biblio devops2

Lilia Kaci

Nada Boussetta

15 juin 2026

1 Fonctionnement global de la recherche

La fonctionnalité de recherche du projet permet aux utilisateurs de trouver des bibliothèques en fonction d'une adresse, d'un rayon de recherche et d'un créneau horaire. Le système repose sur une architecture backend en **Spring Boot** et un frontend en **HTML/JS** (Leaflet pour la cartographie).

L'écosystème de recherche s'appuie sur plusieurs services clés :

- **Géocodage** : Transformation de l'adresse textuelle en coordonnées GPS via l'API publique `api-adresse.data.gouv.fr`.
- **Récupération des données** : Interrogation de l'API Open Data Île-de-France (`data.iledefrance.fr`) pour obtenir la liste des bibliothèques.
- **Mise en cache** : Stockage local (base de données H2 en mémoire par défaut) des données de l'API IDF pour limiter les appels externes et améliorer les performances.
- **Analyse des horaires** : Parsing complexe des chaînes de caractères textuelles (ex : *"du lundi au vendredi de 10h à 18h"*) en objets structurés permettant de déterminer si une bibliothèque est ouverte sur un créneau donné.

2 Différence entre recherche classique et optimisée

Le système propose deux modes de recherche distincts, exposés via deux endpoints séparés dans le backend :

- `/api/bibliotheques/recherche`
- `/api/bibliotheques/itineraire`

2.1 Recherche classique (Cas nominal)

La recherche classique retourne la liste de toutes les bibliothèques situées dans le rayon défini autour de l'adresse cible.

- **Objectif** : Fournir à l'utilisateur un catalogue des options disponibles.
- **Filtrage horaire** : Le système identifie quelles bibliothèques sont ouvertes pendant le créneau demandé.
- **Tri** : Les résultats sont triés par statut d'ouverture (ouvertes en premier), puis par distance croissante, et enfin par note globale décroissante.

2.2 Recherche optimisée (Itinéraire)

La recherche optimisée, déclenchée par le bouton *"Recherche Avancée"*, ne se contente pas de lister les bibliothèques. Elle construit un parcours multi-étapes pour couvrir l'intégralité du créneau horaire demandé.

- **Objectif** : Si aucune bibliothèque n'est ouverte sur l'intégralité du créneau, le système propose de passer d'une bibliothèque à l'autre.
- **Algorithme** : Le service `ItineraireOptimisationService` utilise un **algorithme glouton amélioré**. Il cherche à maximiser le temps passé dans chaque bibliothèque (en choisissant celle qui ferme le plus tard) tout en minimisant la distance de déplacement.
- **Résultat** : Retourne une liste ordonnée d'étapes (Itinéraire), la distance totale à parcourir, et indique si le créneau horaire demandé a pu être complètement couvert.

3 Logiques de Fallback et d'Optimisation

Le système implémente plusieurs stratégies de résilience (*fallback*) pour garantir un résultat pertinent, même en cas de données d'entrée imparfaites ou de défaillance des services externes.

3.1 Stratégie de Fallback du Géocodage

Lorsque l'utilisateur saisit une adresse, le `GeolocationService` applique une stratégie de dégradation gracieuse en 4 étapes :

1. **Normalisation intelligente** : Réorganisation de la chaîne de caractères pour placer la ville en premier.
2. **Tentative nominale** : Appel de l'API avec l'adresse normalisée.
3. **Simplification (Fallback 1)** : Suppression des numéros de rue et des chiffres pour chercher uniquement le nom de la rue.
4. **Ville seule (Fallback 2)** : Extraction de la ville uniquement pour garantir un point de départ approximatif plutôt qu'une erreur bloquante.

3.2 Stratégie de Cache (API Île-de-France)

Pour éviter la latence liée à l'interrogation systématique de l'API IDF et contourner les éventuels timeouts (fixés à **4 secondes** dans `AppConfig`), le système implémente un cache local (`CachedLibraryRepository`) :

- **Peuplement initial** : Si le cache est vide, le système déclenche un appel massif à l'API (`limit=-1`) pour télécharger et persister toutes les bibliothèques.
- **Rafraîchissement asynchrone** : Une tâche planifiée (`@Scheduled(cron = "0 0 3 * * *")`) met à jour ce cache tous les jours à **3h du matin**.
- **Calcul local** : Le filtrage par rayon (distance de Haversine) est effectué localement en mémoire.

3.3 Parsing permissif des horaires

Le `HorairesParser` est conçu pour être hautement tolérant aux variations de format dans l'Open Data :

- Support des plages de jours ("*du lundi au vendredi*") avec résolution automatique des jours intermédiaires.
- Support de multiples formats d'heures (ex : "*10h30-12h00*", "*10 :30-12 :00*", "*10h-12h*").
- En cas d'erreur de parsing, le `HorairesSearchService` capture l'exception et marque la bibliothèque comme **fermée** sans faire planter la requête.

4 Principaux flux du système

1. **Saisie et validation** : L'utilisateur saisit ses critères. Le frontend envoie un **POST** avec le payload **RechercheDTO**. Le **LibraryController** valide (via **@Valid**) que les champs sont présents et le rayon positif.
2. **Résolution de l'origine** : Le **GeolocationService** convertit l'adresse en coordonnées [lat, lon] en appliquant ses stratégies de fallback.
3. **Récupération des candidats** : L'**IleDeFranceLibraryApiService** lit le cache local, le met à jour si nécessaire, et filtre les bibliothèques dans le rayon impart.
4. **Traitement métier** :
 - *En mode classique* : Le **HoraireSearchService** vérifie les intersections horaires, puis le **LibraryService** trie les résultats.
 - *En mode itinéraire* : L'**ItineraireOptimisationService** chaîne les bibliothèques et construit les étapes (**ItineraireEtapeDTO**).
5. **Restitution** : Le frontend reçoit le JSON, génère les cartes interactives Leaflet et affiche les fiches descriptives ou les étapes de l'itinéraire.

4.1 Diagramme de séquence – Recherche classique

Le diagramme ci-dessous détaille l'enchaînement complet des appels pour le mode de recherche classique (**/api/bibliotheques/recherche**), depuis la saisie utilisateur jusqu'à l'affichage de la carte Leaflet.

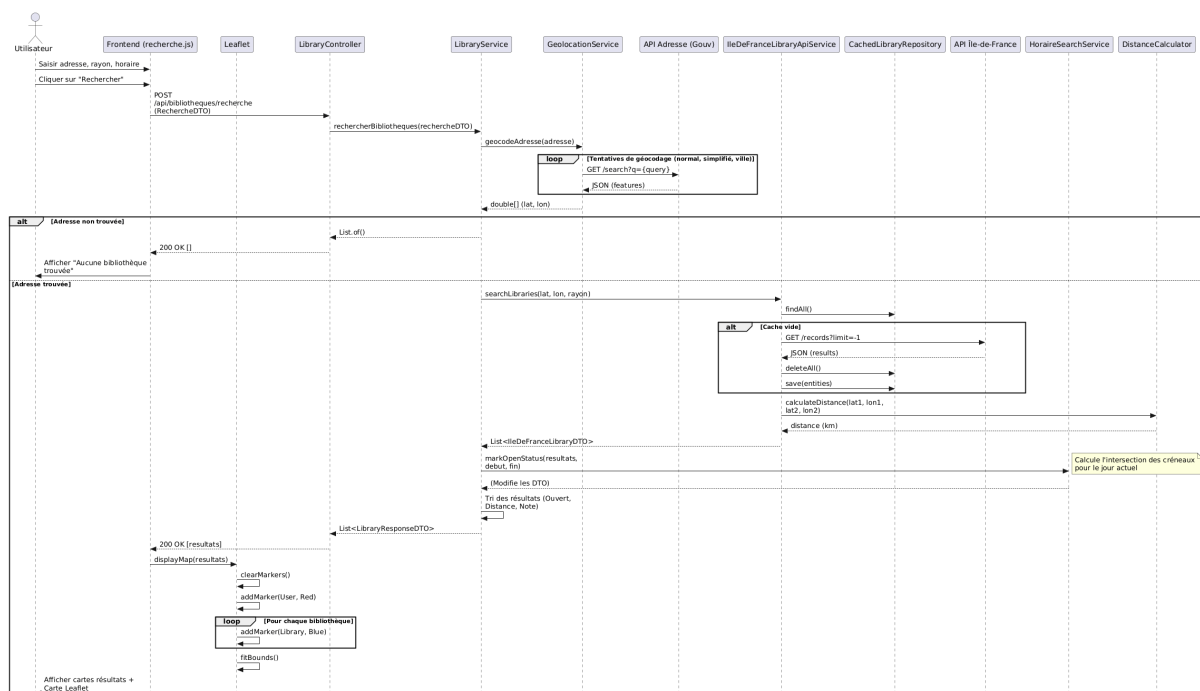


FIGURE 1 – Diagramme de séquence - Recherche simple

4.2 Diagramme de séquence – Recherche optimisée (Itinéraire)

Le diagramme ci-dessous détaille l'enchaînement complet des appels pour le mode itinéraire (/api/bibliotheques/itineraire), incluant la résolution de l'algorithme glouton dans ItineraireOptimisationService.

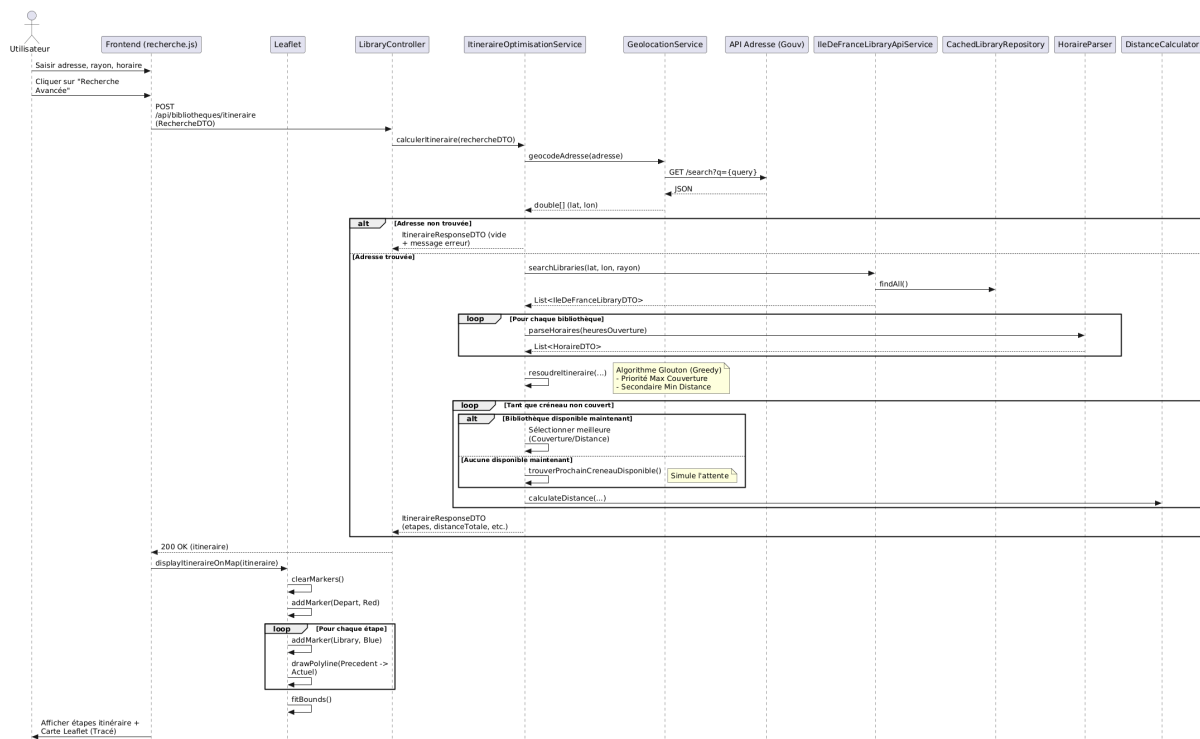


FIGURE 2 – Diagramme de séquence - Recherche optimisée (Itinéraire)